

Experiment 01

Q

~~We are given a 0 indexed~~
~~nums = 0-indexed~~
~~0 ≤ nums[i] ≤ 9~~

Triangular sum - (integer value)

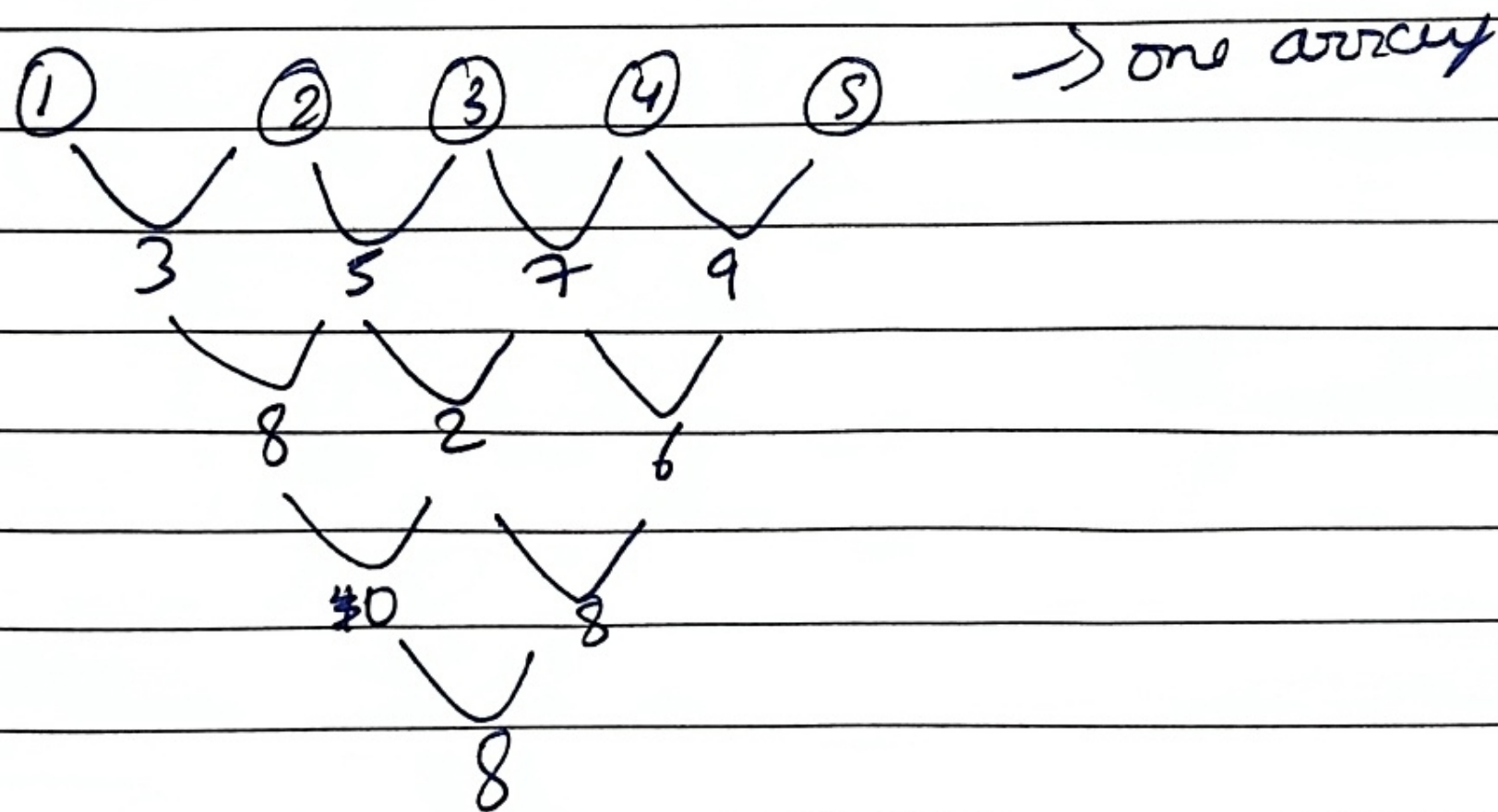
1. $n = 21$ is end of process

otherwise create a new 0 indexed integer array
 newNums of length $n-1$

2. For each index i , $0 \leq i \leq n-1$, assign the value of
 $\text{newNums}[i] = (\text{nums}[i] + \text{nums}[i+1]) \cdot 10$

3. Replace nums with newNums.

4. Repeat entire process



Implementation:

class Solution {

public:

int triangularSum (vector<int> & nums)

{
 int n = nums.size();

while (n > 1)

{
 for (int i = 0; i < n-1; i++)

{
 $\text{nums}[i] = (\text{nums}[i] + \text{nums}[i+1]) \cdot 10$

}

n--;

}
 return nums[0];

}

EXPERIMENT - 07

1 Cheapest flight from source to destination with atmost k-stops.

Algo :

- ① Build the graph $\rightarrow$ convert flight list into adjacency list.
- ② Initialize BFS queue $\rightarrow$ Each element: {current city, total cost so far, stops used}
 $\hookrightarrow$ starts with {source, 0, 0}
- ③ Track minimum cost: create minimumCost[n] which will store cheapest / minimum cost to reach each city.
- ④ BFS Traversal:
 while queue is not empty, we pop the element from front {city, cost, stops}. If stops > k, skip it and for each neighbour,
 $\hookrightarrow$ if city + price < minCost[neighbour city], then update and push into the queue
- ⑤ Return the result
 - If minCost[dst] is still INT-MAX $\rightarrow$ return -1 (not reachable)
 - Else $\rightarrow$ return minCost[dst] (cheapest cost)

Key Idea: Here we use BFS.

Code on next page.

Experiment 07

#include <iostream>

#include <vector>

#include <queue>

using namespace std;

class Solution {

public:

int findCheapestPrice(int n, vector<vector<int>>& flights, int src, int dst, int k) {

for (auto & flight: flights) {

graph[flight[0]].push_back({flight[1], flight[2]});

}

queue<vector<int>> q;

q.push({src, 0, 0});

vector<int> minCost(n, INT_MAX);

minCost[src] = 0;

while(!q.empty()) {

auto current = q.front();

q.pop();

int city = current[0], cost = current[1], stops = current[2];

if (auto & neighbours: graph[city]) {

int nextCity = neighbours.first;

int price = neighbours.second;

if (cost + price < minCost[nextCity]) {

minCost[nextCity] = cost + price;

q.push({nextCity, cost + price, stops + 1});

}

}

}

return minCost[dst] == INT_MAX ? -1 : minCost[dst];

}

Time Complexity: $O(E * k)$ Space Complexity: $O(E + V)$